

1. Problem Statement. Our task here is to take as input the text of a paper and to produce a cleanly formatted paper as output. The output should have one inch margins all around with a page number in the upper right hand corner of each page and date in the upper left hand corner. However, there will be no such header on the first page. We will assume that there are 6 lines vertically and 10 characters to the inch horizontally. To give the user some control over the paper's layout, the formatter will interpret some special commands. These commands will all appear on lines by themselves.

- 1) a blank line — indicates that the next line of text is to begin a new paragraph. The first line of a paragraph should be indented by 0.5 inches. No fewer than two lines of a paragraph should appear at the end of a page.
- 2) **.page** — indicates that the next line of text should begin a new paragraph on the next page.
- 3) **.center** — indicates that the next line is to be centered.
- 4) **.skip** — indicates that a blank line should be placed in the output at this point.
- 5) **.section** — indicates that the next line of text is to be the title of a new section of the paper. The line following that should begin a new paragraph. There should be two blank lines before a section title and one after.
- 6) **.header** — indicates that the next line of text is to be a string centered between the date and the page number in the page headers.
- 7) **.line** — indicates that the next line will be printed literally, even if it begins with a command.

Sentences will be separated by at least two spaces. Lines will have spaces evenly added between words in order to make the lines of uniform length (except for the last line of a paragraph).

2. The Main Program. Our basic strategy is to repeatedly read a line from the standard input and process that line until we reach the end of file. Hence our main program will look like:

```
#include <stdio.h>
#include <stdlib.h>
#include <strings.h>
#include <time.h>
#define INPUT_LINE_LENGTH 256
  <Preprocessor definitions>
  void begin_paragraph(void), break_page(void), center_line(void);
  void skip_line(void), begin_section(void), set_header(void);
  void print_header(void), literal_line(void);
  void process_text_line(char *);
  <Global variables 5>
int main(int argc, char *argv[])
{
  char in_line[INPUT_LINE_LENGTH];
  FILE *fp;
  <Other variables local to main 6>
  <Process command line filename 3>
  <Initialize the globals 7>
  while (gets(in_line) ≠ Λ) {
    <Process input 4>
  }
  <Flush current line 12>
}
```

3. Our input file will either come from the standard input or from a file named on the command line. If the command line has anything but the command itself, then it is the file name.

```
<Process command line filename 3> ≡
if (argc > 1) {
  if ((fp = freopen(argv[1], "r", stdin)) ≡ Λ) fprintf(stderr, "Unable to open file %s\n", argv[1]);
}
```

This code is used in section 2.

4. Processing the input line will take one of two paths. Either we find that the input line is one of the special commands which we'll dispatch to a function of its own or we've got a line of text that we'll pass to a function for processing the line.

```
<Process input 4> ≡
if (in_line[0] ≡ '\0') begin_paragraph();
else if (strcmp(in_line, ".page") ≡ 0) break_page();
else if (strcmp(in_line, ".center") ≡ 0) center_line();
else if (strcmp(in_line, ".skip") ≡ 0) skip_line();
else if (strcmp(in_line, ".section") ≡ 0) begin_section();
else if (strcmp(in_line, ".header") ≡ 0) set_header();
else if (strcmp(in_line, ".line") ≡ 0) literal_line();
else process_text_line(in_line);
```

This code is used in section 2.

5. Processing the formatting commands. We'll start with the handling of the header line. It will be constructed with three parts: the current date, the users's header and the page number. The current date will be kept in a global character array as will be users's header title. The page number part of the string will be constructed as each page is begun.

⟨ Global variables 5 ⟩ ≡
char *cur_date*[14], *user_header*[40];

See also sections 9, 13, and 18.

This code is used in section 2.

6.

⟨ Other variables local to main 6 ⟩ ≡
time_t *now*;
char **cur_time*;

See also section 15.

This code is used in section 2.

7. We must construct the date using the format given to us by *ctime()* which is "Sun_Sep_16_01:03:52_\1973". We want "Sep_16,_1973".

⟨ Initialize the globals 7 ⟩ ≡
time(&*now*);
cur_time = *ctime*(&*now*);
strncpy(*cur_date*, *cur_time* + 4, 6);
cur_date[6] = ',';
strncpy(*cur_date* + 7, *cur_time* + 19, 5);
cur_date[12] = '\\0';
user_header[0] = '\\0';

See also sections 10 and 16.

This code is used in section 2.

8.

```
void set_header(void)
{
    char buf[INPUT_LINE_LENGTH];
    while (gets(buf) ≠ Λ)
        if (buf[0] ≠ '\\0') break;
    strncpy(user_header, buf, 38);
    buf[38] = '\\0';
}
```

9. Everything else deals with producing output. For this purpose, we need a global character array to hold the output line we're currently building.

#define OUTPUT_LINE_LENGTH 65

⟨ Global variables 5 ⟩ +≡
char *cur_line*[OUTPUT_LINE_LENGTH + 1];
int *cur_line_length* = 0;

10.

⟨ Initialize the globals 7 ⟩ +≡
cur_line[0] = '\\0';

11. Now we can address something like centering a line. First, we need to flush the current line. Then we'll get line we need to center. Once we have the line, we'll put it into the current line centered and flush it. To do the calculation we need to know how many spaces make up a line here.

```
void center_line(void)
{
    char buf[INPUT_LINE_LENGTH];
    < Flush current line 12 >
    while (gets(buf) != Λ)
        if (buf[0] != '\0') break;
    sprintf(cur_line, "%*s", (OUTPUT_LINE_LENGTH + strlen(buf))/2, buf);
    < Flush current line 12 >
}
```

12. The < Flush current line 12 > we just used has several things it needs to do. First we won't print an empty current line, for efficiency and so that we don't accidentally end up with more blank lines than we want. If we're already at the bottom of a page, we'll send a form feed and send the page header. Then we put out the line with the appropriate margin. Finally, we clear the line for future use.

```
#define LEFT_MARGIN 10
#define RIGHT_MARGIN 10
#define TOP_MARGIN 6
#define BOTTOM_MARGIN 6
#define PAGE_LINES 66
< Flush current line 12 > ≡
if (cur_line[0] != '\0') {
    if (out_line_num ≥ PAGE_LINES - BOTTOM_MARGIN) {
        < Start a new page 14 >
    }
    printf("%*s%s\n", LEFT_MARGIN, "", cur_line);
    cur_line[0] = '\0';
    cur_line_length = 0;
    out_line_num++;
}
```

This code is cited in section 12.

This code is used in sections 2, 11, 19, 20, 21, 22, 23, and 24.

13. which creates the need for a variable *out_line_num*.

```
< Global variables 5 > +≡
int out_line_num = 0;
```

14.

```
< Start a new page 14 > ≡
putchar('\f');
cur_page_num++;
print_header();
```

This code is used in sections 12, 19, 21, and 22.

15. Since we don't print a header for the first page, we need to print some blank lines for the top margin.

```
< Other variables local to main 6 > +≡
int i;
```

16.

```

⟨Initialize the globals 7⟩ +≡
    for (i = 0; i < TOP_MARGIN; ++i) printf("\n");
    out_line_num = TOP_MARGIN;
    sprintf(cur_line, "UUUUU");
    cur_line_length = 5;

```

17. Printing the header is fairly straightforward. We simply increment the page number and print the date, the user's header and a string like "Page_42". First however, we need to move down all but two of the top margin lines.

```

void print_header(void)
{
    int i, user_hdr_end;
    for (i = 0; i < TOP_MARGIN - 2; ++i) printf("\n");
    printf("%*s", LEFT_MARGIN + strlen(cur_date), cur_date);
    user_hdr_end = (OUTPUT_LINE_LENGTH + strlen(user_header))/2;
    printf("%*s", user_hdr_end - strlen(cur_date), user_header);
    printf("%*s_2d\n\n", OUTPUT_LINE_LENGTH - user_hdr_end - 3, "Page", cur_page_num);
    out_line_num = TOP_MARGIN;
}

```

18. The *cur_page_num* variable is to be global and initialized to 1.

```

⟨Global variables 5⟩ +≡
    int cur_page_num = 1;

```

19. With all of that out of the way, we can return to the matter of the formatting commands. The blank line for new paragraphs is simple.

```

void begin_paragraph(void)
{
    ⟨Flush current line 12⟩
    if (out_line_num ≥ PAGE_LINES - BOTTOM_MARGIN - 2) {
        ⟨Start a new page 14⟩
    }
    sprintf(cur_line, "UUUUU");
    cur_line_length = 5;
}

```

20. A literal line is just as easy.

```

void literal_line(void)
{
    ⟨Flush current line 12⟩
    while (gets(cur_line) ≠ Λ)
        if (cur_line[0] ≠ '\0') break;
    ⟨Flush current line 12⟩
}

```

21. Beginning a new section follows much the same model as for a new paragraph. However, we need two blank lines before the section title which are suppressed if we start a new page. We also need one blank line after and the first two lines of a paragraph. Hence, we must check for needing 6 lines. If fewer are available, we start a new page.

```
void begin_section(void)
{
    char buf[INPUT_LINE_LENGTH];
    ⟨Flush current line 12⟩
    while (gets(buf) ≠ Λ)
        if (buf[0] ≠ '\0') break;
    if (out_line_num ≥ PAGE_LINES - BOTTOM_MARGIN - 6) {
        ⟨Start a new page 14⟩
    }
    else {
        printf("\n\n");
        out_line_num += 2;
    }
    strcpy(cur_line, buf);
    ⟨Flush current line 12⟩
    printf("\n");
    out_line_num++;
    sprintf(cur_line, "UUUUU");
    cur_line_length = 5;
}
```

22. The new page command is fairly easy. The only difficult part is deciding whether to do a blank page if we're already at the beginning of one. One can argue that either way. On the one hand, if we do and the user doesn't want it, then it's a nuisance to go in and take it out. (Especially, if later changes necessitates putting it back in.) On the other hand, if we don't then, there's really no way to get a blank page if one is desired. We chose to come down on the side of utility over convenience.

(The truth is that this type of quandry should lead us to reconsider our requirements and design of the formatting language. However, we're attempting to meet requirement over which we have no control.)

```
void break_page(void)
{
    ⟨Flush current line 12⟩
    ⟨Start a new page 14⟩
}
```

23. We face a similar quandry regarding the skipping of a line in the output. Again, we choose the ability to force blank lines at the top of a page.

```
void skip_line(void)
{
    ⟨Flush current line 12⟩
    printf("\n");
    out_line_num++;
}
```

24. Handling Text. We need to repeatedly find the beginning of a word using the pointer p , then find the end of the word using the pointer q . Next we see if there's enough space in the current output line to hold the word. If so, we copy it over. Put two spaces after each period and colon. If there's not enough room, we justify the line and ship it out.

```

format line x
void process_text_line(char *line)
{
    char *p, *q;
    void justify(void);
    q = line;
    do {
        for (p = q; *p ≡ '␣'; ++p) ;
        if (*p ≡ '\0') return;
        for (q = p; *q ≠ '␣' ∧ *q ≠ '\0'; ++q) ;
        if (q - p > OUTPUT_LINE_LENGTH - cur_line_length) {
            justify();
            ⟨Flush current line 12⟩
        }
        strncpy(cur_line + cur_line_length, p, q - p);
        cur_line_length += q - p;
        if (*(q - 1) ≡ '.' ∨ *(q - 1) ≡ ':') cur_line[cur_line_length++] = '␣';
        cur_line[cur_line_length++] = '␣';
        cur_line[cur_line_length] = '\0';
    } while (*q ≠ '\0');
}

```

25. To justify a line, we need to add extra spaces among the words so that the last non-space character falls at exactly the line length. First we erase any spaces we tacked on at the end. Next, we determine how many extra spaces need to be inserted. Then a count of places to insert them is made, and finally, they are scattered evenly in the line.

```

void justify(void)
{
    int num_add, i, j, k, gaps, in_word, interval;
    int per_gap;
    char temp[OUTPUT_LINE_LENGTH + 1];
    while (cur_line_length > 0 ∧ cur_line[cur_line_length - 1] ≡ '␣') cur_line_length--;
    num_add = OUTPUT_LINE_LENGTH - cur_line_length;
    for (i = 0, gaps = 0, in_word = 0; i < cur_line_length; ++i) {
        if (in_word ∧ cur_line[i] ≡ '␣') {
            gaps++;
            in_word = 0;
        }
        if (cur_line[i] ≠ '␣') in_word = 1;
    }
    if (gaps > 0 ∧ num_add > 0) {
        ⟨Pad the current line 26⟩
        temp[OUTPUT_LINE_LENGTH] = '\0';
        strcpy(cur_line, temp);
    }
}

```

26. Adding the extra spaces can be done a a myriad of ways. We choose to try spreading them out as evenly as possible. To that end, the variable *interval* determines how many gaps there are between (including one) of the additional spaces. The variable *k* keeps track of how far we are between successive additional spaces. It is initialized to *interval*/2 so that the extra spaces aren't heavier on one end of the line or the other. Furthermore, if it starts at 0, then the last added space may not be since we might never get to exactly $k \equiv \text{interval}$.

All this assumes, however, that the number of extra spaces to add is smaller than the number of places to put them. If that's not the case, we add an equal number of spaces to each gap to make it true before applying the distribution algorithm.

```

⟨ Pad the current line 26 ⟩ ≡
  if (num_add > gaps) {
    per_gap = num_add / gaps;
    num_add -= per_gap * gaps;
    for (i = 0, j = 0, in_word = 0; i < cur_line_length; ++i) {
      temp[j++] = cur_line[i];
      if (in_word & cur_line[i] == ' ') {
        in_word = 0;
        for (k = 0; k < per_gap; ++k) temp[j++] = ' ';
      }
      if (cur_line[i] != ' ') in_word = 1;
    }
    strcpy(cur_line, temp);
    cur_line_length += per_gap * gaps;
  }
  interval = gaps / num_add;
  k = interval / 2;
  for (i = 0, j = 0, in_word = 0; i < cur_line_length; ++i) {
    temp[j++] = cur_line[i];
    if (in_word & cur_line[i] == ' ' & num_add > 0) {
      in_word = 0;
      k++;
      if (k == interval) {
        temp[j++] = ' ';
        num_add--;
        k = 0;
      }
    }
    if (cur_line[i] != ' ') in_word = 1;
  }

```

This code is used in section 25.

27. Index.

argc: 2, 3.
argv: 2, 3.
begin_paragraph: 2, 4, 19.
begin_section: 2, 4, 21.
 BOTTOM_MARGIN: 12, 19, 21.
break_page: 2, 4, 22.
buf: 8, 11, 21.
center_line: 2, 4, 11.
ctime: 7.
cur_date: 5, 7, 17.
cur_line: 9, 10, 11, 12, 16, 19, 20, 21, 24, 25, 26.
cur_line_length: 9, 12, 16, 19, 21, 24, 25, 26.
cur_page_num: 14, 17, 18.
cur_time: 6, 7.
fp: 2, 3.
fprintf: 3.
freopen: 3.
gaps: 25, 26.
gets: 2, 8, 11, 20, 21.
i: 15, 17, 25.
in_line: 2, 4.
in_word: 25, 26.
 INPUT_LINE_LENGTH: 2, 8, 11, 21.
interval: 25, 26.
j: 25.
justify: 24, 25.
k: 25.
 LEFT_MARGIN: 12, 17.
line: 24.
literal_line: 2, 4, 20.
main: 2.
now: 6, 7.
num_add: 25, 26.
out_line_num: 12, 13, 16, 17, 19, 21, 23.
 OUTPUT_LINE_LENGTH: 9, 11, 17, 24, 25.
p: 24.
 PAGE_LINES: 12, 19, 21.
per_gap: 25, 26.
print_header: 2, 14, 17.
printf: 12, 16, 17, 21, 23.
process_text_line: 2, 4, 24.
putchar: 14.
q: 24.
 RIGHT_MARGIN: 12.
set_header: 2, 4, 8.
skip_line: 2, 4, 23.
sprintf: 11, 16, 19, 21.
stderr: 3.
stdin: 3.
strcmp: 4.
strcpy: 21, 25, 26.
strlen: 11, 17.
strncpy: 7, 8, 24.
temp: 25, 26.
time: 7.
 TOP_MARGIN: 12, 16, 17.
user_hdr_end: 17.
user_header: 5, 7, 8, 17.

〈Flush current line 12〉 Cited in section 12. Used in sections 2, 11, 19, 20, 21, 22, 23, and 24.
〈Global variables 5, 9, 13, 18〉 Used in section 2.
〈Initialize the globals 7, 10, 16〉 Used in section 2.
〈Other variables local to main 6, 15〉 Used in section 2.
〈Pad the current line 26〉 Used in section 25.
〈Process command line filename 3〉 Used in section 2.
〈Process input 4〉 Used in section 2.
〈Start a new page 14〉 Used in sections 12, 19, 21, and 22.

Trivial Text Formatter

	Section	Page
Problem Statement	1	1
The Main Program	2	2
Processing the formatting commands	5	3
Handling Text	24	7
Index	27	9